

Customer Requirement Discovery Call Transcript

Project: Supplier Onboarding, Duplicate Detection & Risk Scoring

Call Type: Initial requirement discovery

Customer: Global Manufacturing and Services Organization

Participants:

- **Amjad** – Solution Architect
 - **Priya Sharma** – Customer Procurement Operations Manager
 - **James Walker** – Customer Finance Shared Services Lead
 - **Omar Siddiqui** – Customer IT Integration Lead
 - **Linda Chen** – Customer Supplier Master Data Governance Lead
 - **Rachel Adams** – Customer Compliance and Risk Analyst
-

Transcript

Amjad:

Thanks everyone for joining. The purpose of today's call is to understand your supplier onboarding process, the current pain points, and what kind of solution you are expecting. We'll keep this as a discovery session first, and then the implementation team can translate it into requirements and a solution approach.

Priya:

Sure. Our main challenge is supplier onboarding. Different business units request new suppliers through emails, spreadsheets, and sometimes service desk tickets. The process is not standardized, and the data quality is often poor.

Amjad:

When you say supplier onboarding, are you referring to creating completely new suppliers in Oracle Fusion ERP, or also updating existing suppliers?

Priya:

Primarily new supplier requests, but we also get cases where users think they need a new supplier when the supplier already exists. In those cases, we should avoid duplicate creation.

Linda:

Duplicate suppliers are one of our biggest problems. We have suppliers created with slightly different names, different abbreviations, or missing tax details. For example, "ABC Technologies Ltd." and "ABC Tech Limited" may be the same supplier, but they get created separately.

James:

That causes issues in Payables as well. Payments may go to the wrong supplier site, reporting becomes messy, and it becomes difficult to analyze supplier spend.

Amjad:

Understood. So the solution should help detect potential duplicate suppliers before creation.

Linda:

Yes. We don't expect it to be perfect, but it should flag possible duplicates for review.

Rachel:

From a compliance perspective, we also want to identify risky suppliers earlier. Sometimes a supplier request is missing key information, like tax registration, bank details, or business justification. We want the system to highlight those cases instead of letting them move forward unnoticed.

Amjad:

Can you describe the current process from the business user's perspective?

Priya:

A requester sends supplier details to procurement or master data. The information may include supplier name, country, address, contact email, tax registration number, bank information, business unit, supplier type, and reason for onboarding. But there is no single controlled form today.

Amjad:

Would you like a front-end application where users submit supplier requests?

Priya:

Yes. Ideally, users should submit supplier requests through a simple application. We want the form to guide them and capture required details.

Omar:

From IT side, we would prefer Oracle Visual Builder for the UI. We are trying to standardize lightweight internal business apps on Visual Builder.

Amjad:

Got it. And for the backend?

Omar:

Use Oracle ATP as the staging and request database. We don't want the UI to directly create suppliers in Fusion. The data should first be staged, validated, and reviewed.

Amjad:

How should Visual Builder interact with ATP?

Omar:

ORDS would be preferred. We already use ORDS to expose ATP data through REST services in a few internal applications.

Amjad:

Understood. So Visual Builder submits and reads supplier request data through ORDS APIs backed by ATP.

Omar:

Correct.

Amjad:

How do you expect Oracle Integration Cloud to be used?

Omar:

OIC should handle the integration with Oracle Fusion ERP. Once a supplier request passes validation and review, OIC should either create the supplier in Fusion or call the appropriate supplier-related API or process. If Fusion access is not available during the prototype, mock payloads are acceptable, but the integration pattern should be realistic.

James:

We also need status tracking. Users always ask, "What happened to my supplier request?" Today, they send emails to procurement or finance, and no one has a clean answer.

Priya:

Exactly. The requester should see whether the request is submitted, under review, rejected, approved, created in Fusion, or failed during integration.

Amjad:

What kind of approval flow are you expecting?

Priya:

For phase one, we do not need a full enterprise approval workflow. But there should be some review step. Maybe procurement or master data users can approve or reject the request.

Linda:

Yes. The system should not create a supplier automatically just because someone submitted a request.

Rachel:

Especially if there are risk concerns. A risky supplier should require manual review.

Amjad:

Let's talk about the validation rules. What checks should happen before supplier creation?

Linda:

Mandatory fields are important: supplier name, country, supplier type, business unit, contact email, address, and tax registration where applicable.

James:

Payment-related information should also be checked carefully. If bank country is different from supplier country, that should be flagged. It may be valid, but it needs review.

Rachel:

Missing tax registration should increase risk. Also, if the supplier is from a high-risk country or if the business justification is vague, we should flag it.

Priya:

We also want to check if the same tax registration number already exists for another supplier.

Linda:

Yes, tax ID match should be treated as a strong duplicate indicator.

Amjad:

What other duplicate indicators should be considered?

Linda:

Supplier name similarity, tax registration number, email domain, phone number, address similarity, bank account number, and country. Not all of these will always be available, but the more fields that match, the higher the duplicate risk should be.

Omar:

For the prototype, we don't need an extremely advanced matching engine. But it should be more than just exact supplier name matching.

Amjad:

So some combination of exact matching and fuzzy matching would be useful?

Linda:

Yes. For example, ignoring case, punctuation, "Ltd," "Limited," "Inc," and common abbreviations would help.

James:

Also, if the same bank account is already associated with another supplier, that should be a serious warning.

Rachel:

Agreed. That could be a compliance risk.

Amjad:

Should duplicate detection happen before or after user submission?

Priya:

Ideally both. If the user is entering a supplier name and the system can show possible matches early, that would help. But at minimum, duplicate detection should happen after submission before approval.

Omar:

Real-time duplicate check from the form would be nice, but it should not block the whole project. A validation step after submission is acceptable for phase one.

Amjad:

What about AI? How do you want AI involved?

Rachel:

We want AI to help explain risk. Many users don't understand why a supplier is considered risky or why a request is blocked.

Linda:

AI could summarize duplicate risk. For example, "This request may be a duplicate because a supplier with a similar name and same tax registration number already exists."

Priya:

It could also improve the business justification. Sometimes requesters write something vague like "needed for project." AI could suggest that the justification is not sufficient or recommend what information should be added.

Amjad:

Would AI make the decision to approve or reject the supplier?

Rachel:

No. AI should not approve, reject, or create suppliers. It should explain, summarize, and recommend. The final decision should stay with the business reviewer.

James:

That's important. Finance and compliance users need to remain accountable for supplier creation.

Amjad:

What kind of AI output would be useful on the review screen?

Rachel:

A short risk summary would be helpful. Something like: "Medium risk due to missing tax registration and bank country mismatch." Also, recommended action: "Request tax certificate and confirm bank details."

Linda:

For duplicate cases, AI could explain why the supplier looks similar to existing suppliers.

Priya:

And for incomplete requests, it can list what information is missing in plain business language.

Amjad:

Should the AI response be stored?

Omar:

Yes. Store it in ATP with the supplier request, or at least store the generated risk summary and timestamp. We should know what the system recommended at the time of review.

Rachel:

Also, users should be able to regenerate the AI summary if the request data changes.

Amjad:

That makes sense. Should users be able to mark AI suggestions as helpful or not helpful?

Rachel:

That would be nice, but not mandatory for phase one.

Amjad:

Let's discuss risk scoring. Do you already have a defined formula?

Rachel:

Not formally. We have risk factors, but no fixed scoring model yet.

James:

The implementation team can propose something simple and explainable. We do not want a black-box score.

Linda:

Yes. If the risk score is high, the reviewer should know why. For example, duplicate tax ID, missing bank details, high-risk country, or incomplete address.

Amjad:

What risk levels would be useful? Low, Medium, High?

Rachel:

Yes, Low, Medium, High is enough for the prototype. Maybe Critical if the same bank account or tax ID already exists, but we can discuss.

Amjad:

What statuses should the supplier request go through?

Priya:

Something like Draft, Submitted, Validation Failed, Under Review, Approved, Rejected, Submitted to Fusion, Created in Fusion, and Integration Failed.

Omar:

From a technical perspective, we also need to distinguish business validation failure from technical integration failure.

Amjad:

Can you give an example?

Omar:

If supplier name is missing, that is a business validation error. If Fusion API is unavailable or OIC times out, that is a technical error. The support team should be able to see the difference.

James:

That would help a lot. Today, everything becomes "integration failed," and the business does not know what to fix.

Amjad:

Should users be able to retry failed supplier creation?

Omar:

Yes, but only for technical failures or corrected business failures. There should be some retry mechanism from the dashboard.

Linda:

If the issue is duplicate supplier risk, retry should not bypass review.

Amjad:

Understood. What kind of dashboard would you expect?

Priya:

We need a dashboard showing pending requests, requests under review, high-risk requests, duplicate-risk requests, and recently created suppliers.

Linda:

A reviewer should be able to filter by business unit, country, supplier type, requester, request status, risk level, and duplicate risk.

James:

Finance users would like to see requests that are stuck or failed during Fusion creation.

Omar:

And IT needs integration logs: OIC instance ID, request payload reference, response, error message, timestamp, and retry count.

Amjad:

Should the dashboard show possible duplicate suppliers?

Linda:

Yes. On the supplier request detail page, show potential matches from existing suppliers. The reviewer should see why they matched.

Amjad:

For example, similar name, same country, same tax ID, or same bank account?

Linda:

Exactly.

Rachel:

And if the supplier is high risk, show the reasons clearly.

Amjad:

Do you want the system to call external risk services or sanctions lists?

Rachel:

That would be ideal eventually, but not required for this prototype. For now, use internal risk indicators and maybe a configurable high-risk country list.

Omar:

Yes, let's not depend on third-party APIs for phase one.

Amjad:

What data should be captured in the supplier request?

Priya:

Supplier name, supplier type, country, address, contact person, contact email, phone, business unit, requester, business justification, product or service category, expected annual spend, tax registration number, and bank information if available.

James:

Bank information may not always be available at initial request, but if provided, it should be validated.

Linda:

We also need site information. A supplier can have multiple sites, so at least one supplier site or intended business unit should be captured.

Amjad:

Should the prototype support multiple supplier sites?

Linda:

It would be good to support at least one site. Multiple sites would be nice, but not mandatory unless the team has time.

Priya:

At minimum, one supplier with one site is acceptable for phase one.

Amjad:

What about attachments, such as tax certificates or registration documents?

Priya:

Nice to have, but not mandatory. If the team can capture document metadata, that is enough.

Rachel:

The system can flag missing documents, even if actual upload is not implemented.

Amjad:

Understood. How much data volume should we assume for the prototype?

Omar:

A few hundred existing supplier records and maybe 50 to 100 supplier requests is enough. The design should not be hardcoded to five records, though.

Linda:

We can provide sample supplier master data if needed, but mock data is acceptable as long as it represents real-world cases.

Amjad:

What real-world cases should be included in the sample data?

Linda:

One clean new supplier, one duplicate by exact tax ID, one duplicate by similar name, one supplier with missing tax registration, one supplier with bank country mismatch, one with incomplete address, and one with same bank account as an existing supplier.

Rachel:

Also include one request with a vague business justification and high expected spend. That should be flagged for review.

James:

And one case where Fusion supplier creation fails due to missing site or invalid business unit mapping.

Amjad:

That's helpful. Should the system update the requester when the supplier is created?

Priya:

For now, showing status in the application is enough. Email notifications are not required.

Amjad:

What about role-based access?

Omar:

Basic role separation is enough for the prototype. Requesters can create and view their own requests. Reviewers can review and approve or reject. Admin or support users can see integration logs and retry failed requests.

Priya:

That works.

Amjad:

What should happen if a reviewer determines the request is a duplicate?

Linda:

They should be able to reject the request and reference the existing supplier. Or mark it as duplicate and close it.

James:

It would be useful if the requester can see which existing supplier should be used instead.

Amjad:

Should the solution allow merging supplier records?

Linda:

No, not for phase one. Merging is a more controlled process and should not be part of this prototype.

Amjad:

What about updating existing suppliers?

Priya:

Not required initially. We can keep the scope focused on new supplier onboarding.

Amjad:

Let's talk about integration with Oracle Fusion ERP. What is your expectation after approval?

Omar:

After approval, OIC should transform the staged supplier request into the required Fusion payload and submit it. The system should capture Fusion response, supplier number if created, and any error messages.

James:

If creation is successful, the supplier request should show the Fusion supplier number.

Linda:

If Fusion rejects it, the error should be visible to the reviewer or support user.

Amjad:

Should Fusion be the source of truth for supplier master?

Linda:

Yes. ATP is only staging and tracking. Once the supplier is created in Fusion, Fusion is the system of record.

Amjad:

Should existing supplier data be synchronized from Fusion into ATP for duplicate checking?

Omar:

Yes, at least some supplier reference data should be available in ATP. It could be extracted periodically from Fusion through OIC.

Linda:

Duplicate detection will not work well if the application only checks against new requests. It must compare against existing supplier master data too.

Amjad:

So there are two data flows: one to sync existing suppliers into ATP, and another to submit approved new suppliers to Fusion?

Omar:

Correct, that is the expected pattern.

Amjad:

For the prototype, if access to Fusion supplier master is limited, can existing suppliers be mocked in ATP?

Omar:

Yes. Mock existing supplier data is acceptable, but the integration design should still show how Fusion supplier data would be synchronized.

Amjad:

Are there any compliance constraints around bank data?

Rachel:

Yes. Bank data should be handled carefully. For the prototype, full bank account numbers do not need to be displayed. Masking would be preferred.

James:

Show only the last four digits if needed.

Amjad:

Good point. Should the duplicate detection use the masked value or original value?

Omar:

For the prototype, the team can use a simplified approach. But they should explain how sensitive data would be handled in a real implementation.

Rachel:

Exactly. We don't want sensitive bank information exposed unnecessarily.

Amjad:

What are your expectations for documentation?

Omar:

We need a short design document, integration flow, data model, API list, validation rules, risk scoring logic, AI prompt approach, and known limitations.

Priya:

Also include a demo script. We want to see the solution from requester submission to reviewer decision to Fusion creation or failure.

Amjad:

For the final demo, what would you like to see?

Priya:

A requester submits a supplier request from the Visual Builder app.

Linda:

Then the system identifies that it may be a duplicate because a similar supplier already exists.

Rachel:

Then AI explains the risk and recommends what the reviewer should verify.

James:

Then show a clean supplier request being approved and submitted to Fusion, with the supplier number returned.

Omar:

Also show one integration failure and how it appears in the support dashboard.

Amjad:

So the demo should include both successful and failed scenarios.

Priya:

Yes. We do not want a happy-path-only demo.

Amjad:

Any expectations around timelines?

Priya:

We are looking for a prototype in about three weeks. It does not need to be production-ready, but it should be realistic and end-to-end enough to demonstrate value.

Omar:

The main thing is that the team should use the Oracle stack properly: Visual Builder, ATP, ORDS, OIC, and Fusion ERP or realistic Fusion-like mock payloads.

Rachel:

And AI should be included, but safely. It should not approve suppliers, reject suppliers, or create suppliers automatically.

Linda:

Also, duplicate detection should be explainable. If the system says duplicate risk is high, the reviewer should see the matching fields.

James:

And statuses should be clear. Users should know whether the request failed because of bad data, because it is a duplicate risk, or because the integration failed.

Amjad:

Understood. Let me summarize what I heard. You need a supplier onboarding solution where users submit supplier requests through a Visual Builder application. The data should be staged in ATP and exposed through ORDS. OIC should be used to synchronize existing supplier data from Fusion and to submit approved suppliers to Fusion. The system should validate data, detect potential duplicates, calculate risk, and allow reviewers to approve, reject, or request corrections. AI should provide risk explanations and recommended actions, but it should not make final decisions or directly update Fusion on its own. The solution should include dashboards, status tracking, integration logs, retry handling, and a clear demo with both success and exception scenarios.

Priya:

Yes, that captures it well.

Linda:

Please make sure duplicate detection is treated as a key part of the requirement, not just a minor validation.

Rachel:

And risk explanation is important. Users will not trust a score unless they understand the reasons behind it.

Omar:

From IT, make sure the design separates staging, validation, review, and Fusion submission. Also include proper error handling.

James:

From finance, show how this prevents duplicate suppliers and payment-related issues.

Amjad:

Great. We'll take this input and convert it into requirements, assumptions, open questions, and a proposed solution approach.

Additional Notes Mentioned During the Call

- Visual Builder should be used for the supplier request and review application.

- ATP should store supplier requests, validation results, risk scores, possible duplicate matches, integration logs, and Fusion responses.
- ORDS should expose REST APIs between Visual Builder and ATP.
- OIC should be used for Fusion ERP integration.
- Existing Fusion supplier master data should be available in ATP for duplicate checking.
- Mock supplier master data is acceptable if Fusion access is limited.
- AI should explain risk, duplicate reasons, and recommended actions.
- AI must not approve, reject, or create suppliers automatically.
- Duplicate detection should consider supplier name, tax ID, country, address, email domain, phone, and bank information where available.
- Bank data should be masked or handled carefully.
- The solution should distinguish business validation errors from technical integration errors.
- Reviewers should be able to approve, reject, mark duplicate, or request correction.
- The prototype should include success, duplicate-risk, high-risk, and integration-failure scenarios.
- The expected prototype timeline is approximately three weeks.